

AI봇 및 코딩게임 관련

AI 활동 보고서

— CodinGame 글로벌 플랫폼 활동 정리 —

제출자	박진우 (goodpju2008@gmail.com)
플랫폼	CodinGame (전 세계 개발자 200만 명 이상 참여, 글로벌 코딩·AI Bot 경진 플랫폼)
활동 범위	AI Bot Programming / Algorithm (2개 트랙)
활동 기간	2025~2026
작성일	2026-05-13

문서 구성

구분	파일
메인	본 문서 — AI봇 및 코딩게임 관련 AI 활동 보고서.docx
[별첨1]	AI_Bot_Ultimate_TicTacToe_개발 활동 보고서 (Ultimate Tic-Tac-Toe AI Bot — 한국 1위)
[별첨2]	AI_Bot_Olympbits_개발 활동 보고서 (Olympbits AI Bot — 한국 1위)
[별첨3]	Algorithm_Clash_of_Code_활동 보고서 (Clash of Code — 전세계 32위)

1. 개요

1.1 보고서 목적

이 보고서는 제가 글로벌 코딩 및 AI Bot 경진 플랫폼인 CodinGame에서 직접 수행한 AI 관련 활동 내역과 그 결과를 정리한 문서입니다. 활동은 크게 두 가지 트랙에 걸쳐 있는데, ① AI Bot Programming(직접 만든 AI 봇끼리의 대결)과 ② Algorithm(실시간 알고리즘 문제 해결)입니다. 그 중에서도 특히 AI Bot Programming 트랙에서는 제가 직접 설계하고 구현한 AI Bot 2종이 한국 1위를 달성했습니다.

1.2 활동 결과 요약

CodinGame은 AI Bot Programming, Algorithm(Clash of Code) 등 다양한 트랙을 운영하고 있고, 모든 참가자가 동일한 룰 위에서 자기가 만든 AI·알고리즘으로 점수를 다투는 방식입니다. 사람의 주관적인 평가가 끼어들 여지가 없고, 매치메이킹·랭킹이 전부 자동으로 정량 산출되기 때문에 객관적인 지표라고 볼 수 있습니다.

저는 위 두 분야 모두에서 글로벌 상위권에 들었고, 특히 AI Bot Programming 분야에서는 단순히 정해진 알고리즘 문제를 푸는 것이 아니라 제가 직접 설계한 AI(MCTS, Expectimax, 휴리스틱 평가함수 등)가 전 세계 다른 플레이어의 AI와 실제로 대결해서 거둔 결과라는 점에서 의미가 크다고 생각합니다.

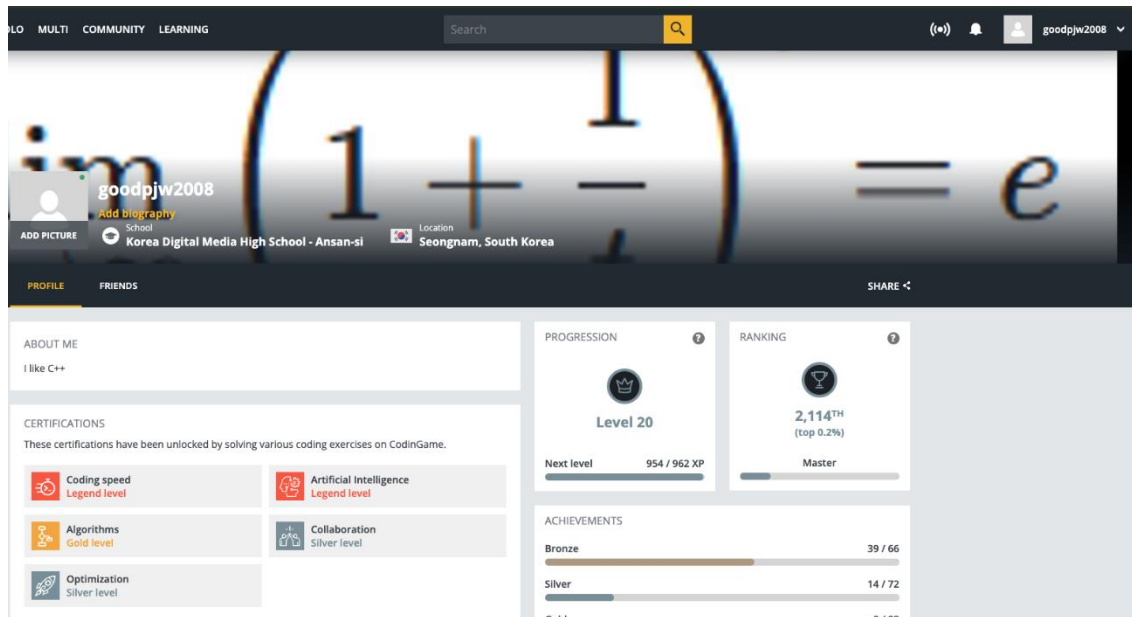
핵심 성과: CodinGame 공식 Certification 시스템에서 **Artificial Intelligence 분야 Legend 등급(최상위)**과 **Coding Speed 분야 Legend 등급(최상위)**을 동시에 획득했고, 전체 계정 글로벌 랭킹 2,114위 (Top 0.2%, Master Tier)를 기록 중입니다. (상세는 2장 참조)

분야	글로벌 순위	비율	한국 비율	핵심 기술
AI Bot Programming	2,534위 / 약 25만+	Top 1%	Top 0.9% (한국)	MCTS, Expectimax, Minimax + α - β 가지치기
Algorithm (Clash of Code)	32위 / 906,316명	Top 0.1%	—	실시간 알고리즘 문제 해결

2. 종합 프로필 및 공식 인증 등급 (★ 핵심 성과)

CodinGame은 활동 결과를 누적해 분야별 Certification 등급(Bronze→Silver→Gold→Legend)과 전체 글로벌 랭킹·Tier(Apprentice→Master→Legend)로 표시합니다.

2.1 종합 프로필 캡처



[그림] CodinGame 종합 프로필 — AI · Coding Speed 동시 Legend 등급

2.2 핵심 결과

지표	결과
전체 글로벌 랭킹	2,114위 — Top 0.2%
전체 Tier	Master

2.3 분야별 공식 Certification 등급

분야	등급	비고
Artificial Intelligence	★ Legend	최상위 등급 — AI 분야 만점 인증
Coding Speed	★ Legend	최상위 등급 — 코딩 속도 만점 인증

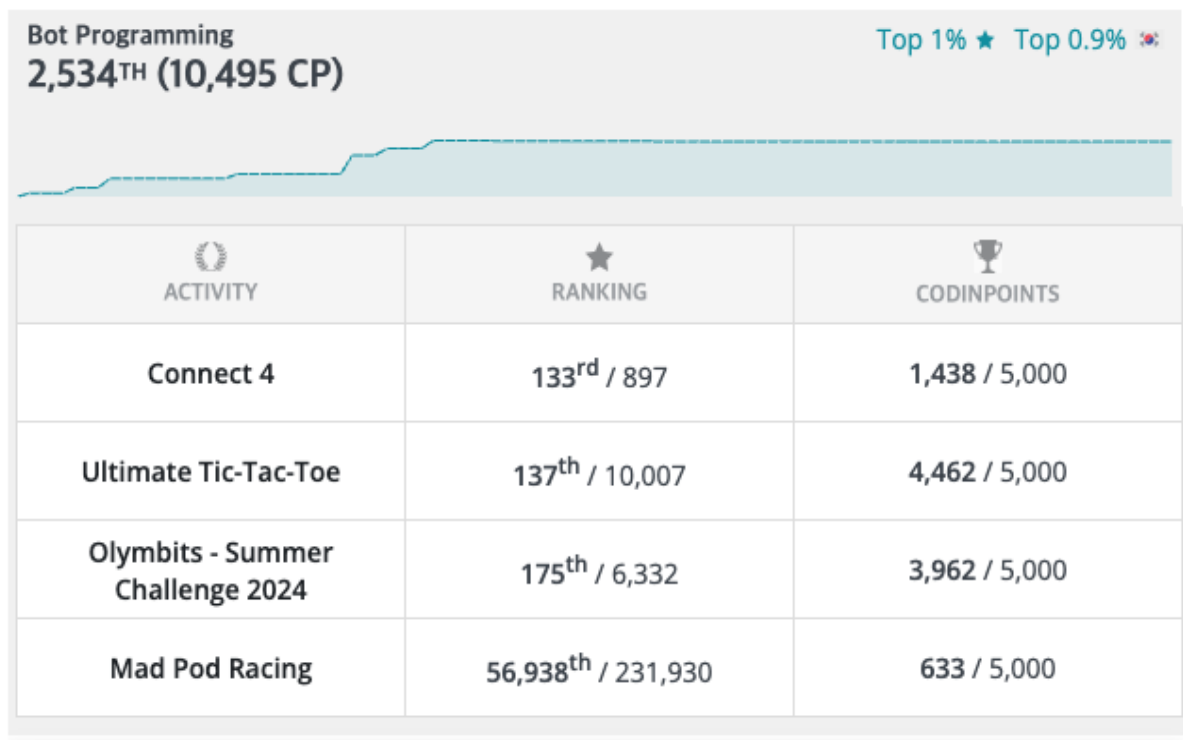
※ Legend는 CodinGame Certification 시스템의 최상위입니다.

3. AI Bot Programming — 자작 AI 봇 대결 트랙 (★ 핵심)

3.1 트랙 개요

CodinGame의 Bot Programming 트랙은 참가자가 직접 작성한 AI 봇끼리 게임으로 대결시키는 방식입니다. 게임의 룰만 주어지고, 어떻게 의사결정을 할지는 100% 참가자가 직접 설계해야 합니다. 응답 시간 제한도 빠듯합니다. 보통 첫 턴은 1초, 그 뒤로는 매 턴 100ms 안에 모든 판단을 끝내야 하기 때문에 탐색 효율과 알고리즘 최적화가 절대적으로 중요합니다. 또 단순히 "정답이 있는 알고리즘 문제"가 아니라, 상대 AI의 전략을 예상하고 거기에 맞춰 반격하는 메타 AI 설계가 진짜 핵심이라고 느꼈습니다.

3.2 트랙 종합 결과



[그림] Bot Programming 트랙 종합 순위

- 글로벌 누적 2,534 위 (10,495 CP) — 전 세계 상위 1%
- 한국 내 상위 0.9%
- 다수 종목에서 한국 1 위 차지 (아래 참조)

3.3 종목별 결과

종목	글로벌 순위	한국 순위	AI 핵심 기술
Ultimate Tic-Tac-Toe	137위 / 10,007명 (Top 1.4%)	한국 1위	MCTS + Minimax 종료국면 탐색
Olympbits — Summer Challenge 2024	175위 / 6,332명 (Top 2.8%)	한국 1위	Expectimax + α - β + Softmax 상대 모델링
Connect 4	133위 / 897명	—	게임 트리 탐색
Mad Pod Racing	56,938위	—	(입문 단계 봇)

3.3.1 Ultimate Tic-Tac-Toe — 한국 1 위

[그림] Ultimate Tic-Tac-Toe 한국 1 위 리더보드

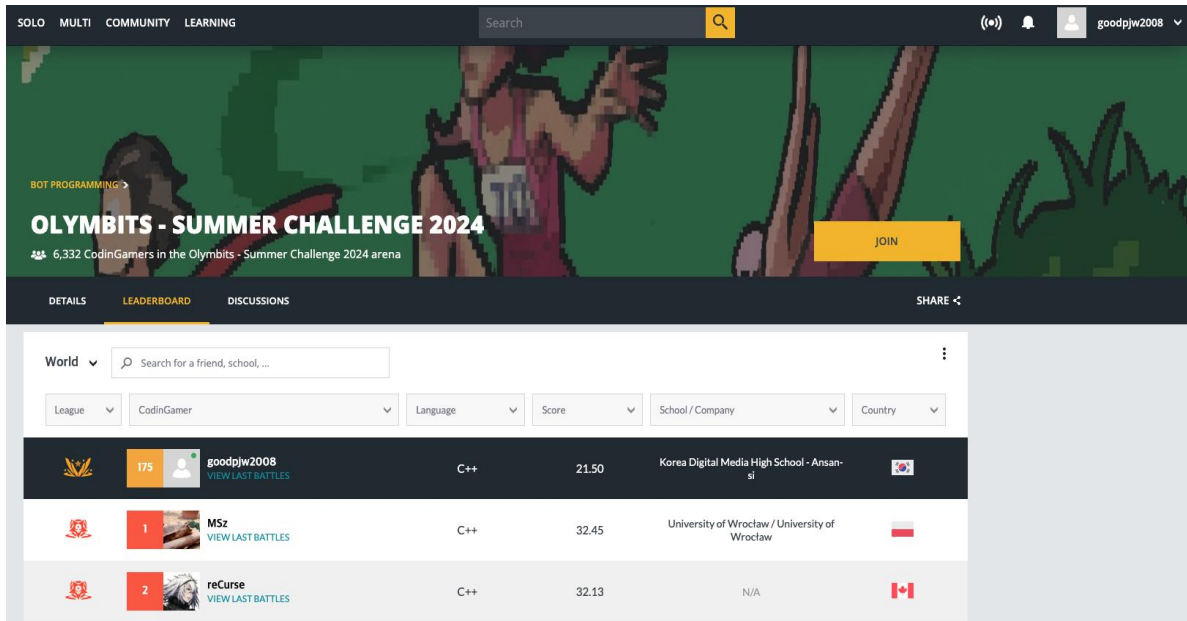
공식 리더보드(증빙): <https://www.codingame.com/multiplayer/bot-programming/tic-tac-toe/leaderboard>

리더보드 캡처를 보면 제 계정 goodpjuw2008이 한국 부문 최상단에 위치한 것을 확인할 수 있습니다.

- 소스코드: source_ai_bot_ultimate_tictactoe/ultimate_tictactoe.cpp (334 줄)
- 핵심: MCTS + Minimax 하이브리드 — 100ms 안에 수만 회 시뮬레이션 + 종반부 미니맥스 검증

상세 분석은 [별첨1] AI_Bot_Ultimate_TicTacToe_개발 활동 보고서를 참고해주세요.

3.3.2 Olymbits — Summer Challenge 2024 — 한국 1 위



Rank	Username	Score	Language	School / Company
175	goodpjuw2008	21.50	C++	Korea Digital Media High School - Ansan-si
1	MSz	32.45	C++	University of Wroclaw / University of Wroclaw
2	reCurse	32.13	C++	N/A

[그림] Olymbits Summer Challenge 2024 한국 1 위 리더보드

공식 리더보드(증빙): <https://www.codingame.com/multiplayer/bot-programming/summer-challenge-2024-olymbits/leaderboard>

CodinGame의 글로벌 챌린지인 Summer Challenge 2024에서 제 계정 goodpjuw2008이 한국 부문 1위를 차지했습니다.

- 소스코드: source_ai_bot_olymbits/olymbits.cpp (588 줄)
- 핵심: Expectimax + α - β + Sigmoid 상대 모델링 — 4 종목 동시 진행 환경에서 깊이 12 탐색

상세 분석은 [별첨2] AI_Bot_Olymbits_개발 활동 보고서를 참고해주세요.

3.4 AI Bot Programming 트랙 활동 의의

- 단순 알고리즘 문제 풀이가 아니라, 스스로 의사결정을 하는 자율 AI를 설계할 수 있는 능력을 보여준다고 생각합니다.

게임 룰만 주어진다면 상태 모델링 → 평가함수 설계 → 탐색 전략 선택 → 시간/메모리 최적화까지 처음부터 끝까지 직접 설계해야 합니다. 그리고 그 결과로 두 종목(Ultimate Tic-Tac-Toe, Olymbits) 모두 한국 1위를 달성했습니다.

3.5 두 봇이 보여주는 AI 역량 종합 매핑

앞에서 설명한 두 봇이 각각 어떤 AI 역량을 보여주는지 비교 정리해보면 다음과 같습니다.

AI 핵심 역량	Ultimate Tic-Tac-Toe	Olymbits
트리 탐색 알고리즘 설계	MCTS 4단계 표준 구현	Expectimax + α - β 깊이 12
탐색-활용 균형(Exploration-Exploitation)	UCB1, C=0.4 튜닝	빔 폭 동적 조정
확률적 상대 모델링	(MCTS 시뮬레이션 자체가 확률적)	Sigmoid/Softmax 정책
다목적 최적화	(단일목적)	Log-기댓값 변환, 4종목 동시 최적화
도메인 지식 임베딩	비트보드 + 사전계산 룩업	게임별 DP/거리/선형결합 평가 함수
알고리즘 하이브리드화	MCTS + Minimax 종반 보완	Expectimax + Beam Search + 휴리스틱
실시간 추론 최적화 (100ms 제약)	트리 재사용·GC, 룩업, SIMD, splitmix64	Undo 스택, 무브 오더링, SIMD
하이퍼파라미터 튜닝	탐색 상수 C=0.4, midlim=40, middep=2	게임별 온도 T=3/150/8/20, 깊이 12

3.6 학습 기반 AI(딥러닝)와의 관계

제가 만든 봇들은 딥러닝 기반이 아니고 고전 AI 탐색(Search-based AI) 계열입니다. 그런데 알파고 같은 현대 강화학습 AI의 핵심 골격을 공유하고 있다는 점에서 의미가 있다고 생각합니다.

본 봇 기법	대응되는 현대 AI 시스템
MCTS + UCB1	AlphaGo, AlphaZero 의 의사결정 코어
Sigmoid 기반 상대 모델링	강화학습 Soft Actor-Critic의 Boltzmann 정책

Log-기댓값 최대화	Policy Gradient의 log-likelihood objective
빔 서치	LLM 디코딩 / Seq2Seq 추론
휴리스틱 평가함수	알파고 Value Network의 hand-crafted 전조
시간 예산 기반 적응형 추론	LLM의 추론 시간 스케일링(Inference-time scaling)

정리하면, 저는 신경망 학습 없이도 AlphaZero가 의존하는 동일한 알고리즘 골격(MCTS + 평가 함수)을 직접 설계·구현해본 셈이고, 나중에 학습 기반 모델로 확장할 때도 이 토대 위에서 신경망을 평가함수·정책 네트워크로 자연스럽게 끼워 넣을 수 있다고 생각합니다.

4. Algorithm 트랙 (Clash of Code) — 세계 Top 0.1%

Clash of Code 32 ND (4,999 CP)			Top 0.1% ★
ACTIVITY	RANKING	CODINPOINTS	
Clash of Code	32 nd / 906,316	4,999 / 5,000	

[그림] Algorithm 트랙 종합 순위

- 글로벌 **32 위 / 906,316 명 (Top 0.0035%)** — 세계 0.1% 이내
- CodinPoints **4,999 / 5,000** — 만점에 1 점 모자란 사실상 만점
- Clash of Code: 짧은 시간(15 분 이내) 안에 알고리즘 문제를 푸는 실시간 경쟁 모드
- 90 만 명 규모의 참가자 풀에서 32 위는 사실상 최상위권
- 평가 척도: 정확성, 속도, 코드 간결성

이 트랙은 AI 봇 자체가 아니라, AI/ML 시스템을 구현할 때 꼭 필요한 알고리즘 사고력과 구현 속도를 보여주는 지표라고 생각합니다. AI 엔지니어링의 기본기를 검증해주는 기록이라고 봅니다.

상세 분석은 [별첨3] Algorithm_Clash_of_Code_활동 보고서를 참고해주세요.

5. 종합 평가

5.1 AI 역량 측정 결과 매핑

역량 영역	측정 근거
AI 의사결정 시스템 설계	Ultimate Tic-Tac-Toe(MCTS+Minimax 하이브리드), Olymbits(Expectimax+상대 모델링) — 모두 한국 1위
탐색 알고리즘 깊이 있는 이해	MCTS, Minimax, α - β 가지치기, Expectimax를 게임 특성에 맞게 선택·하이브리드화
확률 모델링/베이지안 사고	시그모이드 기반 상대 행동 확률 추정, 메달 기대값 로그 변환
실시간 추론 성능 최적화	100ms 내 수만 회 시뮬레이션 — 비트보드, SIMD, 룩업테이블, 트리 재활용
알고리즘 기본기	Clash of Code 글로벌 32위 (Top 0.1%)

이 보고서에서의 결과물은 제가 직접 만든 AI 봇들이 글로벌 25만+ 참가자 규모의 플랫폼에서 경쟁력을 가지고 상위권에 있다는데 의미가 있다고 생각합니다. 또한 최고급 AI 개발자가 되기 위한 알고리즘 기본기를 갖추기 위해 많은 노력을 하였습니다.

5.2 활동 결론

제가 만든 AI봇들은 AI를 재미있게 공부한 결과물입니다. 그리고 그 기반에는 알고리즘 문제 해결에 대한 열정이 있었다고 생각합니다. 앞으로도 AI에 대한 재미와 기반 기술에 대한 열정을 가지고 계속 경험을 쌓고 더욱 노력하겠습니다. 다음은 활동에 사용된 AI 관련 기법입니다.

1. 상태 모델링 (비트보드 / Undo 스택)
2. 탐색 알고리즘 설계 (MCTS / Expectimax + α - β)
3. 확률·통계적 의사결정 (UCB1 / Sigmoid 정책)
4. 다목적 최적화 (Log-기댓값 변환)
5. 도메인 지식 임베딩 (게임별 휴리스틱·DP)
6. 하이퍼파라미터 튜닝 (탐색상수·온도·깊이)
7. 실시간 추론 최적화 (시간예산·SIMD·트리재활용)

6. 첨부

6.1 관련 보고서

파일	내용
[별첨1] AI_Bot_Ultimate_TicTacToe_개발 활동 보고서	Ultimate Tic-Tac-Toe AI Bot 단독 개발 활동 보고서
[별첨2] AI_Bot_Olymbits_개발 활동 보고서	Olymbits AI Bot 단독 개발 활동 보고서
[별첨3] Algorithm_Clash_of_Code_활동 보고서	Algorithm 트랙(Clash of Code) 단독 활동 보고서

[별첨 1]

AI Bot Ultimate Tic-Tac-Toe

개발 활동 보고서

— CodinGame Bot Programming 트랙 · 한국 1 위 —

개발자	박진우 (goodpju2008@gmail.com)
플랫폼	CodinGame — 글로벌 코딩 및 AI Bot 경진 플랫폼
종목	Ultimate Tic-Tac-Toe (Bot Programming 트랙)
구현 언어	C++
소스코드	source_ai_bot_ultimate_tictactoe/ultimate_tictactoe.cpp
공식 리더보드	https://www.codingame.com/multiplayer/bot-programming/tic-tac-toe/leaderboard
활동기간	2025~2026
작성일	2026-05-13

1. 개요

1.1 보고서 목적

이 보고서는 CodinGame의 Bot Programming 트랙 종목 중 하나인 Ultimate Tic-Tac-Toe에서 동작하는 AI Bot에 대해, 제가 설계를 어떻게 했고 어떤 알고리즘을 적용했으며 실제 성능 결과는 어땠는지 정리한 개발 활동 보고서입니다. 이 AI 봇은 전 세계 10,007명이 참가하는 같은 종목에서 글로벌 137위(상위 1.4%), 한국 1위를 기록했습니다.

1.2 한 줄 요약

MCTS(Monte Carlo Tree Search)와 Negamax+ α - β 가지치기를 상황별로 결합한 하이브리드 의사 결정 AI로, 100ms 시간 제약 안에서 매 턴 수만~수십만 회의 시뮬레이션과 종반부 외통수 검증을 동시에 수행해 자율적으로 최적 수를 결정합니다.

2. 개발 배경 및 문제 정의

2.1 종목 룰

일반 틱택토(3×3)와는 다르게 훨씬 복잡한 "보드 안에 보드가 있는" 변형 게임입니다.

- 전체 보드: **9 개의 작은 보드(3×3 매크로) × 각 작은 보드 9 칸(3×3 마이크로) = 총 81 칸**
- 한 턴에 한 칸을 두며, **내가 둔 마이크로 좌표가 다음 상대가 둘 수 있는 매크로 보드를 결정합니다.** (예: 내가 작은 보드의 우상단에 두면, 상대는 다음 턴에 매크로 보드의 우상단 작은 보드 안에서만 둘 수 있어요.)
- 작은 보드 하나에서 한 줄(가로/세로/대각) 완성 → 해당 매크로 칸을 차지
- 9 개 매크로 칸 중 한 줄 완성 → 전체 게임 승리
- **시간 제약:** 첫 턴 1 초, 이후 매 턴 100ms 이내에 수를 출력해야 합니다.

2.2 AI 관점의 도전 과제

특성	수치	함의
분기수 (branching factor)	평균 6~9, 최악 81	일반 틱택토(3) 대비 매우 큼 → 완전 탐색 불가
게임 깊이	최대 81 수	깊은 트리, Minimax 단독으로는 한계
상태 공간	약 $3^{81} \approx 10^{38}$	사실상 모든 상태 메모이제이션 불가
평가함수 정의	매우 어려움	매크로 보드 점령, 다음 강제 보드, 위치 가치가 복잡하게 얽힘
시간 예산	100ms / 턴	단위 시간당 시뮬레이션 횟수가 곧 성능

위와 같은 특성 때문에 단일 알고리즘만으로는 한계가 분명합니다. MCTS의 확률적 탐색과 Minimax의 결정론적 정밀도를 결합하는 하이브리드 접근이 거의 필수적이라고 판단했습니다.

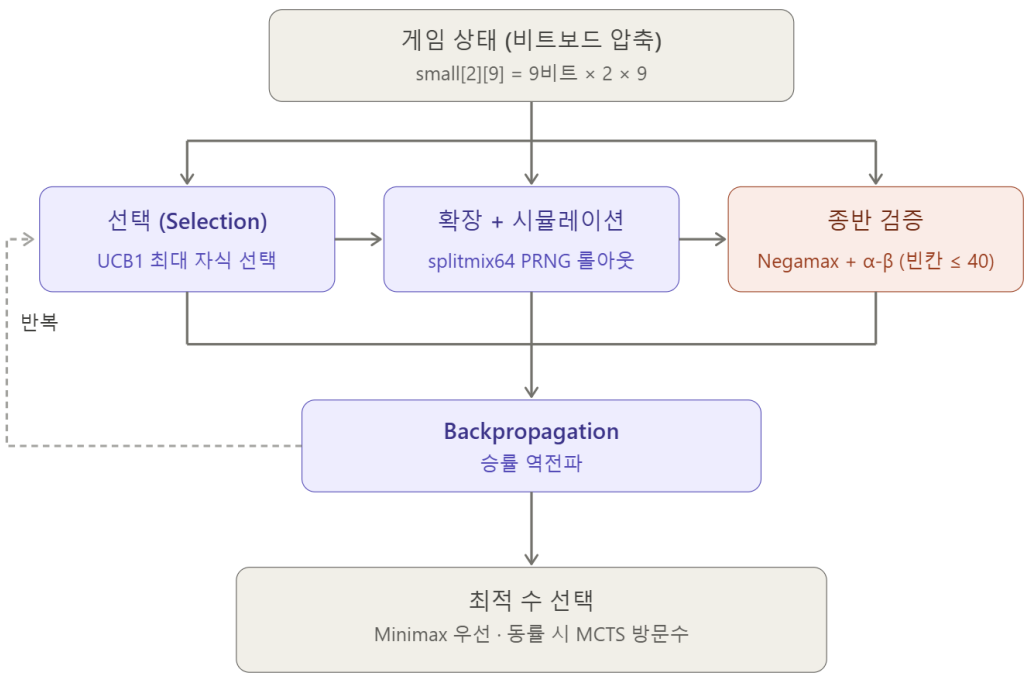
2.3 개발 목표

목표	정량 지표	결과
100ms 안에 의미 있는 의사결정 수행	매 턴 10,000+ MCTS 반복	✓ 달성
MCTS의 종반 외통수 누락 약점 보완	종반부 깊이 2 Minimax 검증	✓ 달성
메모리 한계 내에서 무한 게임 지속	정적 노드 풀 + GC	✓ 달성
글로벌 상위권 진입	Top 5% 이내	✓ 달성 (Top 1.4%)
한국 내 최고 성능	한국 1위	✓ 달성

3. 시스템 아키텍처

3.1 전체 데이터 흐름

아래 다이어그램은 매 턴 의사결정이 어떤 순서로 진행되는지 정리한 것입니다.



3.2 핵심 설계 결정

결정	근거
MCTS를 주 알고리즘으로 선택	평가함수 정의 난이도 높은 종목으로 무작위 시뮬레이션 기반
MCTS + Minimax 하이브리드	순수 MCTS의 종반 외통수 누락 약점 결정론적 미니맥스로 보완
비트보드 + 사전계산 록업	시뮬레이션 핫루프의 상태 전이·승패 판정을 $O(1)$ 로 단축
정적 노드 풀 + GC	동적 할당 오버헤드 제거, 메모리 한계 내 무한 게임 지속 가능
트리 재사용	매 턴 누적된 시뮬레이션 정보를 폐기하지 않고 활용
SplitMix64 PRNG	표준 rand() 대비 10~30배 빠른 시뮬레이션
SIMD/AVX2 컴파일 옵션	핫루프 자동 벡터화

4. 핵심 알고리즘 설계

4.1 핵심 알고리즘 ① — Monte Carlo Tree Search (MCTS)

MCTS는 알파고(AlphaGo)의 핵심 기반이 된 트리 탐색 알고리즘으로, 평가함수가 명확히 정의되지 않는 게임에서 수많은 무작위 시뮬레이션을 통해 통계적으로 좋은 수를 찾는 기법입니다. 제가 만든 봇은 표준 4단계를 모두 구현했습니다.

단계	코드 위치	설명
① Selection	ultimate_tictactoe.cpp:210-224	루트에서 시작해 자식 노드 중 UCB1 값이 최대인 노드를 재귀적으로 따라 내려감
② Expansion	ultimate_tictactoe.cpp:225-233	미탐색 수 중 하나를 무작위 선택해 자식 노드로 추가
③ Simulation	ultimate_tictactoe.cpp:234-236	확장된 노드에서 게임 종료까지 무작위 플레이(Rollout)
④ Backpropagation	ultimate_tictactoe.cpp:241-249	결과(승=1, 무=0.5, 패=0)를 경로상 모든 노드에 역전파, 턴 부호 반전

4.2 핵심 알고리즘 ② — UCB1 (Upper Confidence Bound)

MCTS의 핵심은 "이미 좋은 수를 더 깊이 파고들지(활용)" vs "아직 안 가본 수를 시도할지(탐색)"의 균형을 맞추는 일입니다.

$$UCB1(child) = (win/vis) + C \times \sqrt{(\ln(parent.vis) / vis)}$$



- 코드: ultimate_tictactoe.cpp:213-220 — $val = tree[c].win*iv[vis] + vval*rsq[vis]$
- 탐색 상수 **C = 0.4** (이론값 $\sqrt{2} \approx 1.41$ 보다 활용 비중을 키운 수동 튜닝)
- $iv[n]=1/n$, $rsq[n]=1/\sqrt{n}$, $slg[n]=\sqrt{\ln(n)}$ 을 **110 만 크기 룩업 테이블**로 사전 계산해서 UCB1 1 회 평가가 곱셈 2 번으로 단축됩니다.

4.3 핵심 알고리즘 ③ — MCTS + Minimax 하이브리드 (★ 차별점)

순수 MCTS는 종반의 외통수(forced win/loss)를 놓치기 쉬운 약점이 있습니다. 무작위 시뮬레이션이 정확한 외통수 라인을 항상 만들어내지 못하기 때문입니다. 그래서 종반부에 이를 보완하도록 설계했습니다.

- 남은 빈 칸이 **40 개 이하**가 되면 (ultimate_tictactoe.cpp:254) 모드 전환
- 모든 후보 수에 대해 **깊이 2 의 Negamax + α - β 가지치기** (probe 함수, ultimate_tictactoe.cpp:116-129)로 확정승/패를 우선 판별
- Minimax 값이 동률일 때만 MCTS 방문수로 동률 해소
- = Search-based AI 두 기법의 장점을 **상황별로 결합한 메타-알고리즘 설계**

아래는 실제 코드 발췌입니다 (ultimate_tictactoe.cpp L262-273):

```
for(int i = 0; i < cnt; i++){
    int mv = arr[i];
    tictactoe nxt = cur;
    nxt.apply(mv);
    int chk = -probe(nxt, middep-1, -1, 1); // ← Minimax  $\alpha$ - $\beta$ 
    if(chk > bestchk || (chk == bestchk && rvis[mv] > bestrate)){
```



```

bestchk = chk;                // ← 우선순위 1: Minimax 결과

bestrate = rvis[mv];          // ← 우선순위 2: MCTS 방문수

bestmv = mv;

...

}

}

```

5. 구현 상세

5.1 트리 재사용 — 점진적 정보 누적

MCTS는 매 턴 수만~수십만 번의 시뮬레이션을 누적하는데, 매 턴 트리를 새로 만들면 그동안 쌓인 정보가 전부 버려져 버립니다. 그래서 두 가지 메커니즘으로 이를 해결했습니다.

- **applyreal()** — **루트 이동** (ultimate_tictactoe.cpp:171-196): 실제 게임에서 한 수가 두어지면 트리를 폐기하는 대신 해당 수에 대응되는 자식 노드를 새 루트로 승격. 이전 턴까지 누적된 통계량을 그대로 활용.
- **recycle()** — **트리 가비지 컬렉션** (ultimate_tictactoe.cpp:135-170): 정적 노드 풀(110 만)의 90% 이상이 차면 현재 루트의 후손 노드만 남기고 압축(in-place compaction). 메모리 한계 안에서 탐색을 무한히 지속할 수 있도록 했습니다.

5.2 상태 표현(State Representation) — 비트보드

AI 탐색 속도 = 단위 시간당 시뮬레이션 횟수이기 때문에, 상태 복사·전이·승패 판정이 가장 중요한 핫루프입니다. 도메인 지식을 활용해서 상태를 비트로 압축했습니다.

- 9 칸 작은 보드 → unsigned short(9 비트)로 표현 (ultimate_tictactoe.cpp:10)
- 9 개의 승리 라인을 8 개의 비트마스크 $h[8] = \{0x007, 0x38, 0x1C0, 0x49, 0x92, 0x124, 0x111, 0x54\}$ 로 표현
- 512 가지 보드 상태 전체에 대해 승리 여부 $win[512]$ 를 사전 계산 → **O(1) 승패 판정**
- 빈 칸 인덱스도 $zcnt[], zpos[][]$ 로 사전 계산 → 합법수 생성 O(빈칸 수)
- 무승부 판정도 $_\text{builtin_popcount}(big[0])$ vs $big[1]$ 로 매크로 보드 점령 수 비교

5.3 시뮬레이션 품질 — SplitMix64 PRNG

Monte Carlo의 통계적 정확도는 난수 품질과 생성 속도 양쪽에 의존합니다. 표준 rand()는 느리고 분포도 좋지 않아서 SplitMix64를 사용했습니다.

- 코드: ultimate_tictactoe.cpp:63-72
- 비트 시프트와 곱셈 3 번만으로 균등 분포의 64 비트 난수를 생성하고, rand() 대비 **10~30 배 빠른 속도**를 보여줍니다.
- 시뮬레이션 단계 *getrandvalid()*에서 호출되어 합법수를 균등 확률로 선택합니다.

5.4 시간 예산 관리

- 첫 턴 **996ms**, 이후 **96ms** 로 동적 조정 (ultimate_tictactoe.cpp:317-322)
- 매 **32 회 반복마다 한 번씩** 시각 비교(*it & (chkstride-1)*, ultimate_tictactoe.cpp:204-206) → 시각 측정 syscall 오버헤드 최소화

5.5 저수준 성능 최적화

최적화	코드 위치	효과
#pragma GCC optimize("Ofast,unroll-loops,inline")	L2	컴파일러 최적화 최대
#pragma GCC target("avx2,fma,sse,...")	L3	SIMD 명령어(AVX2/FMA) 활성화
정적 노드 풀 tree[max_nodes]	L112	동적 할당 제거
inline + 템플릿	전체	하트로 인라이닝

5.6 적용 AI 기법 총정리

#	기법	AI 분야 분류
1	Monte Carlo Tree Search (MCTS)	강화학습-인접 탐색 (AlphaGo 계열)
2	UCB1 (Upper Confidence Bound)	다중 무장 강도(MAB) / 탐색-활용 균형
3	Negamax + α - β 가지치기	고전 게임 트리 탐색

4	MCTS-Minimax 하이브리드 ★	멀티 알고리즘 결합 / 메타-알고리즘 설계
5	트리 재사용 & 트리 GC	점진적 학습 / 메모리 관리
6	비트보드 + 사전계산 룩업	도메인 지식 임베딩
7	SplitMix64 PRNG	몬테카를로 시뮬레이션 품질
8	앵커시간 기반 시간 예산	실시간 추론 (Inference-time scaling)
9	SIMD/AVX2 컴파일 최적화	하드웨어 가속

6. 성능 측정 결과

6.1 글로벌 리더보드 성적

The screenshot shows the 'Ultimate Tic-Tac-Toe' leaderboard on the codinggame.com website. The page features a dark theme with a 3D background of tic-tac-toe boards. The leaderboard is titled 'World' and includes a search bar for finding friends or schools. The top three users are listed with their avatars, usernames, scores, and countries. The user 'goodpju2008' is highlighted in the top position, showing a score of 20.71 and representing 'Korea Digital Media High School - Ansan-si'.

Rank	Username	Score	Country
137	goodpju2008	20.71	Korea Digital Media High School - Ansan-si
1	MrSubZero	33.60	N/A
2	Fancheng	33.13	N/A

[그림] Ultimate Tic-Tac-Toe 리더보드 — 한국 1위

공식 리더보드(증빙): <https://www.codinggame.com/multiplayer/bot-programming/tic-tac-toe/leaderboard>

리더보드 캡처에서 제 계정 goodpju2008이 한국 부문 최상단에 있고, 전체 10,007명 중 137등에 위치 하고 있습니다.

지표	결과	비고
글로벌 순위	137위 / 10,007명	상위 1.4%
한국 순위	1위	
CodinPoints	4,462 / 5,000	만점 대비 89.2%
평가 방식	봇 간 직접 대결 매치메이킹 + ELO	객관적·정량적 평가

CodinGame의 Bot Programming 트랙은 AI 봇끼리 직접 대결하는 방식이라, 평가관의 주관이 개입되지 않는 객관적·정량적 결과입니다.

6.2 실행시간 성능 특성

단계	시간 예산	봇 동작
첫 턴	1,000ms	996ms 사용 — 초반 트리 대규모 구축
일반 턴	100ms	96ms 사용 — 트리 재사용으로 효율적 확장
시간 체크 주기	32회/반복	syscall 오버헤드 최소화

6.3 정량적 최적화 효과

최적화 기법	단축 효과
비트보드 + win[512] 록업	승패 판정 $O(N) \rightarrow O(1)$
iv[]/rsq[] 록업 테이블	UCB1 평가에서 나눗셈·√ 호출 제거 → 곱셈 2번
SplitMix64 PRNG	rand() 대비 10~30배 빠름
트리 재사용	매 턴 트리 처음부터 재구축 회피 → 이전 누적 통계 유지
SIMD/AVX2	하루프 자동 벡터화
정적 노드 풀	malloc/free 오버헤드 0

6.4 달성 목표 검증

2.3에서 정의했던 개발 목표 전부 달성했습니다.

- 100ms 안에 의미 있는 의사결정 수행
- MCTS 종반 외통수 누락 보완 (Minimax probe)
- 메모리 한계 내 무한 게임 지속 (정적 풀 + GC)
- 글로벌 상위권 진입 (Top 1.4%)
- 한국 내 최고 성능 (1 위)

7. 현대 AI 시스템과의 대응 관계

제가 만든 AI 봇은 딥러닝 기반이 아니라 고전 AI 탐색(Search-based AI) 계열입니다. 그렇지만 알파고 같은 현대 강화학습 AI의 핵심 골격을 그대로 공유하고 있습니다.

본 봇 기법	대응되는 현대 AI 시스템
MCTS + UCB1	AlphaGo, AlphaZero 의 의사결정 코어
트리 재사용	AlphaZero의 self-play 트리 캐싱
휴리스틱 평가함수 (Minimax probe)	알파고 Value Network의 hand-crafted 전조
시간 예산 기반 적응형 추론	LLM의 추론 시간 스케일링(Inference-time scaling)

정리하면, 제가 만든 AI 봇은 신경망 학습 없이도 AlphaZero가 의존하는 동일한 알고리즘 골격 (MCTS + 평가함수)을 직접 설계·구현한 결과물이라고 볼 수 있습니다. 나중에 학습 기반 모델로 확장한다고 해도 이 토대 위에서 신경망을 평가함수·정책 네트워크로 교체하는 방식으로 자연스럽게 발전시킬 수 있다고 생각합니다.

8. 결론

이 보고서는 Ultimate Tic-Tac-Toe AI Bot의 개발 의사결정과 성능 측정 결과를 정리한 문서입니다. 제가 만든 AI 봇이 보여준 기술적 가치는 다음과 같다고 생각합니다.

1. 자율 의사결정 AI 설계 능력: 룰만 주어진 상태에서 상태 모델링 → 알고리즘 선택 → 평가함수 설계 → 성능 최적화까지 전 과정 직접 수행했습니다.
2. 알고리즘 하이브리드화 능력: MCTS의 통계적 강점과 Minimax의 결정론적 정밀도를 상황별 트리거(빈칸 ≤ 40)로 결합한 메타-알고리즘 설계.
3. 실시간 추론 최적화: 100ms 제약 안에서 의미 있는 의사결정을 위해 비트보드, 록업테이블, SIMD, 트리 재활용 등 다층적 최적화 적용.
4. 객관적 검증 통과: 동일 룰 위에서 글로벌 10,007명과 직접 대결해 글로벌 Top 1.4%, 한국 1위 달성.

8.1 향후 확장 가능성

- 현재 무작위로 진행하는 MCTS를 신경망 기반으로 바꿔 AlphaZero 구조로 개선
- Minimax probe의 깊이를 시간 예산에 따라 동적으로 조정
- 정책-가치 분리 학습으로 시뮬레이션 단계 자체를 신경망으로 대체

9. 첨부

파일	내용
<code>source_ai_bot_ulimate_tictactoe/ultimate_tictactoe.cpp</code>	봇 전체 소스 (C++)

[별첨 2]

AI Bot Olymbits

개발 활동 보고서

— CodinGame Summer Challenge 2024 · 한국 1 위 —

개발자	박진우 (goodpju2008@gmail.com)
플랫폼	CodinGame — 글로벌 코딩 및 AI Bot 경진 플랫폼
종목	Olymbits — Summer Challenge 2024 (CodinGame 연례 글로벌 챌린지)
구현 언어	C++
소스코드	source_ai_bot_olymbits/olymbits.cpp
공식 리더보드	https://www.codingame.com/multiplayer/bot-programming/summer-challenge-2024-olymbits/leaderboard
활동기간	2025~2026
작성일	2026-05-12

1. 개요

1.1 보고서 목적

이 보고서는 CodinGame이 2024년 여름에 개최한 글로벌 챌린지 Olymbits (Summer Challenge 2024) 종목에서 동작하는 AI Bot에 대해, 제가 어떤 설계 결정을 내렸고 어떤 알고리즘 구조를 적용했으며 실제 성능 결과는 어땠는지를 정리한 개발 활동 보고서입니다. 이 봇은 전 세계 6,332명이 참가한 같은 종목에서 글로벌 175위(상위 2.8%), 한국 1위를 기록했습니다.

1.2 한 줄 요약

Expectimax + α - β 가지치기(깊이 12)를 기반으로, Sigmoid 기반 상대 행동 확률 모델링과 Log-기댓값 변환을 통한 다목적 → 단일목적 변환을 결합한 의사결정 AI입니다. 4개 미니게임이 동시에 진행되고 3명의 플레이어가 동시에 행동하는 복합 환경에서, 100ms 시간 제약 안에 자율적으로 최적의 행동을 결정합니다.

2. 개발 배경 및 문제 정의

2.1 종목 룰

Olymbits는 일반적인 봇 대결 종목과는 차원이 다른 복합 동시-다목적 게임입니다.

- **4 개의 미니게임을 동시 진행:** 허들 레이스(Hurdle Race), 양궁(Archery), 스케이트(Roller Speed Skating), 다이빙(Diving)
- **3 명의 플레이어가 동시에 한 가지 행동(LEFT / UP / RIGHT / DOWN)을 선택**
- 선택한 한 가지 행동이 **4 개 미니게임 모두에 동시 작용** (예: RIGHT 를 누르면 허들에서는 오른쪽 이동, 양궁에서는 조준 이동 등 동시에 적용됩니다)
- 최종 점수 = **4 개 종목 메달 가치의 곱(Multiplicative Score)**
- 시간 제약: 첫 턴 1 초, 이후 매 턴 100ms 이내

2.2 AI 관점의 도전 과제

특성	결과	함의
행동 공간	4 (LEFT/UP/RIGHT/DOWN)	작아 보이지만 동시-동시-동시 적용
다목적성	4개 게임 동시 평가 필요	단일 목적함수로 환원 필요
동시 행동	3명 동시 결정	표준 미니맥스 부적합
점수 결합 방식	메달 가치의 곱	한 종목 0이면 전체 0 (발산 위험)
각 게임의 상태 차원	게임마다 별도(위치/위험도/스턴 등)	통합 평가함수 설계 난이도 ↑
시간 예산	100ms / 턴	12레벨 탐색을 위해선 분기 축소 필수

이 종목은 단순 게임 트리 탐색으로는 한계가 분명해합니다. (1) 동시-행동 게임의 확률적 상대 모델링, (2) 다목적 → 단일목적 변환, (3) 게임별 도메인 지식 평가함수, (4) 알파-베타 효율 극 대화의 4가지 요소를 모두 결합해야 한다고 판단했습니다.

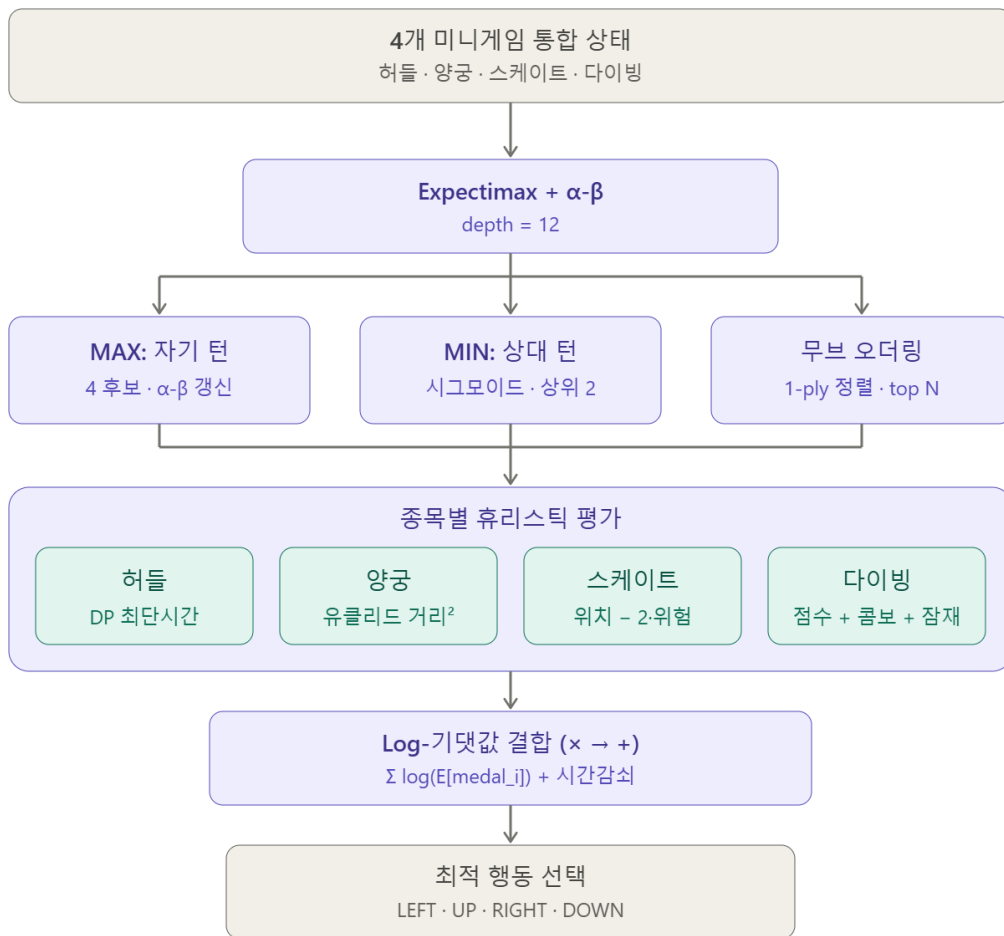
2.3 개발 목표

목표	정량 지표	결과
100ms 안에 깊이 12 게임 트리 탐색 수행	depth ≥ 12 Expectimax	✓ 달성
동시 행동 게임의 상대 모델링	Sigmoid 정책으로 확률화	✓ 달성
급셈 목적함수의 α - β 가지치기 적용 가능	Log-기댓값 변환	✓ 달성
4개 미니게임 각각에 도메인 특화 평가	게임별 휴리스틱	✓ 달성
글로벌 상위권 진입	Top 5% 이내	✓ 달성 (Top 2.8%)
한국 내 최고 성능	한국 1위	✓ 달성

3. 시스템 아키텍처

3.1 전체 데이터 흐름

아래 다이어그램은 매 턴 의사결정이 어떤 순서로 진행되는지 전체 진행 과정의 흐름을 나타냅니다.



3.2 핵심 설계 결정

결정	근거
Expectimax + α - β 를 주 알고리즘으로 선택	봇이 3개인 게임이라 최적/최악의 수만 고려하는 Minimax 부적합, 확률적 상대 모델링 필요
Sigmoid 기반 상대 행동 확률 모델링	"상대도 가능한 좋은 수를 둔다"는 Boltzmann 정책 — RL의 Soft Policy와 동치
Log-기댓값 변환	메달 점수가 곱셈 결합 → 작은 종목이 0이면 전체 발산. 로그 합산으로 변환해 α - β 적용 가능 + 기하평균 효과
게임별 휴리스틱 분리 설계	각 미니게임의 상태 구조와 보상 구조가 완전히 다름
무브 오더링 + 빔 폭 동적 조정	깊이 12 탐색을 100ms에 맞추기 위한 가지치기 효율 극대화

Undo 스택 기반 상태 복원	매 노드 깊은 복사 회피, 12레벨 탐색에 필수
온도 게임별 튜닝	4개 게임의 코스트 스케일이 다름 (허들 3, 양궁 150, 스케이트 8, 다이빙 20)
SIMD/AVX2 컴파일 옵션	하루프 자동 벡터화

4. 핵심 알고리즘 설계

4.1 핵심 알고리즘 ① — Expectimax + α - β 가지치기

표준 미니맥스는 상대를 결정론적 최적 플레이어로 가정하는데, Olymbits는 3명이 동시에 행동하는 게임이라 결정론적 상대 모델이 부정확합니다. 그래서 Expectimax 변형을 사용했습니다.

- 코드: olymbits.cpp:409-531
- **MAX 노드(자기 턴)**: 4개 후보 수를 모두 평가해 최댓값 선택 + α - β 가지치기 (olymbits.cpp:447-464)
- **MIN 노드(상대 턴)**: 상대가 비례적으로 합리적인 행동을 한다고 가정 → 상대 평가함수 기준 상위 2개 수에 대해 최솟값 (olymbits.cpp:506-529)
- 최대 깊이 12 레벨까지 탐색 (mxdep=12, olymbits.cpp:8) — 한 턴 = 3개의 결정 노드이므로 실질적으로 4턴 앞을 내다봅니다.

4.2 핵심 알고리즘 ② — Sigmoid 기반 상대 행동 확률 모델링 (★ 차별점)

순수 미니맥스가 부적합한 동시 행동 게임에서, "상대가 자기에겐 유리한 수를 둘 확률"을 시그모이드로 모델링했습니다.

$$P(\text{상대 A가 상대 B를 이길 확률}) = \sigma((\text{cost_A} - \text{cost_B}) / T) \\ = 1 / (1 + \exp(-(\text{cost_A} - \text{cost_B})/T))$$

- 시그모이드: olymbits.cpp:51-53
- 각 미니게임마다 별도 구현(prob 함수): 허들 63-65 / 양궁 153-155 / 스케이트 201-203 / 다이빙 363-365

온도(Temperature)는 게임별로 별도 튜닝했습니다:

게임	온도 T	튜닝 근거
허들	3	이산적 위치 차이 — 작은 스케일
양궁	150	거리 제곱 단위라 스케일이 매우 큼
스케이트	8	위치 + 위험도 + 스텝 — 중간 스케일
다이빙	20	점수 + 잠재력 — 누적 점수라 큰 스케일

시그모이드($\sigma(x) = 1/(1+e^{-x})$)는 Boltzmann 분포의 2-action 특수형이고, 강화학습의 Soft Policy(Soft Actor-Critic 등)와 동치입니다. 즉 상대를 "결정론적이 아니라 비례적으로 합리적"으로 모델링한 것이고, 동시 행동 게임에서 실제 인간/봇 상대의 거동에 더 가깝다고 생각합니다.

4.3 핵심 알고리즘 ③ — Log-기댓값 변환 (★ 차별점)

Olympbits의 최종 점수는 4종목 메달 가치의 곱입니다. 곱셈 형태는 α - β 가지치기와 잘 맞지 않고, 작은 메달이 0이면 전체가 0이 되는 발산 위험도 있습니다. 그래서 다음과 같이 변환했습니다.

```
maximize  $\prod E[\text{medal}_i]$ 
```

```
 $\equiv \text{maximize } \log \prod E[\text{medal}_i] = \sum \log E[\text{medal}_i]$ 
```

- 각 종목별 `score()` 함수가 반환하는 값이 이미 $\log(\dots)$ (olymbits.cpp:68-74)
- 합산된 로그 기댓값을 평가함수로 사용 (olymbits.cpp:411)
- 결과적으로 **로그-기댓값 최대화** $\equiv$ **기하평균 최대화** — 한 종목이 망해서 0이 되는 위험을 자연스럽게 피하고, 4 종목에 균형 있게 메달이 분배되도록 유도됩니다.

아래는 실제 코드 발췌입니다 (olymbits.cpp L411):

```
ld ret = hud.score(me,0) + arc.score(me,1)
      + skt.score(me,2) + divv.score(me,3)
      - ( hud.score((me+1)%3,0)+arc.score((me+1)%3,1)+...
          + hud.score((me+2)%3,0)+arc.score((me+2)%3,1)+... ) / 2;
// 자기 로그-기댓값 - 상대 두 명 평균 로그-기댓값
```

5. 구현 상세

5.1 무브 오더링 (Move Ordering) — α - β 효율 극대화

α - β 가지치기는 좋은 수를 먼저 평가할수록 가지치기 효과가 커집니다. 제 봇은 깊이마다 1-ply 예비 평가 → 정렬 → 깊은 탐색 패턴을 사용합니다.

1. 4 개의 모든 수를 일단 1-ply 깊이로 평가 (olymbits.cpp:436-446)
2. 점수 내림차순 정렬 (sort(lst, lst+4, greater<pli>()), olymbits.cpp:447)
3. 깊이가 깊어질수록 후보 폭을 줄임: $4 - \min(2, \text{depth}/3)$ (olymbits.cpp:448) — 빔 서치(Beam Search) 변형

깊이	탐색 후보 수
0~2	4개 (모두)
3~5	3개
6+	2개

이렇게 하면 트리 폭이 지수적으로 폭발하는 것을 막으면서, 좋은 수에 시간을 집중할 수 있습니다.

5.2 게임별 휴리스틱 평가함수 설계

각 미니게임마다 도메인 지식 기반의 코스트 함수를 따로 설계했습니다.

5.2.1 허들 (Hurdle Race)

- DP 로 골인까지의 최단 시간 사전 계산 (olymbits.cpp:122-139)
- $dp[i]$ = 위치 i 에서 골인까지의 최소 비용 (장애물 '#'을 만나면 +4 페널티 반영)
- 코스트: $-dp[position] - \text{stun_timer}$ → 골인 가까울수록, 스톤 적을수록 우위

5.2.2 양궁 (Archery)

- 표적 중심까지의 유클리드 제곱거리 ($\sqrt{\text{생략}}$ 으로 가속): $\text{cost} = -(x^2 + y^2)$
- 단조 변환이므로 거리 자체를 쓰는 것과 동등하지만 $\sqrt{\text{호출}}$ 1 회 제거

5.2.3 스케이트 (Roller Speed Skating)

- 선형 결합 평가: $position - 2 \times risk - 2 \times stun_timer$
- 위험도와 스톤 가중치를 위치 가치의 2 배로 페널티 부여
- 깊이 3 이상에서는 랜덤 시뮬레이션으로 전환 (olymbits.cpp:224-249) — 행동의 효과가 다음 턴 상태에 의존하므로 깊은 탐색에서는 결정론보다 기댓값 추정이 정확

5.2.4 다이빙 (Diving)

- 콤보 기댓값 + 남은 턴 잠재력: $point + combo \times (\text{남은 턴})/2 + \text{남은 턴} \times (\text{남은 턴} + 1)/2$
- 현재 점수 + 콤보 유지 시 예상 추가 점수 + 남은 턴 동안의 등차수열 합으로 잠재력 평가
- 콤보가 끊기지 않도록 행동 선택을 유도

5.3 동적 메달 가치 (Temporal Discounting)

이미 자신/상대가 딴 메달은 시간이 지날수록 가치를 감쇠시켰습니다.

```
medalvalue[t][i] = 1/4 * (turns3) + 3 * gold + 1 * silver + (-0.001) * bronze
```

- 코드: olymbits.cpp:550-556
- 게임 후반으로 갈수록 기본 보너스가 감쇠하여, 남은 게임의 결과에 더 큰 가중치가 실리도록 설계
- 동(bronze) 보상을 -0.001 로 매우 작게 줘서 동메달보다는 은메달 이상을 노리는 전략을 학습
- 강화학습의 할인계수(γ)와 보상 설계(reward shaping)에 대응되는 개념

5.4 상태 되돌리기(Undo Stack) — 트리 탐색 메모리 최적화

12레벨 깊이 탐색에서 매 노드마다 4종목 상태를 깊은 복사하면 메모리·시간이 폭발해버립니다. 그래서 변경분만 스택에 기록하는 방식으로 풀었습니다.

- *trackint*, *trackpii*, *tracktuple* 커스텀 스택 (olymbits.cpp:13-50)
- 각 게임의 `move()`는 `push`, `undo()`는 `pop` 으로 $O(1)$ 상태 복원
- `std::vector` 대신 정적 배열 기반 스택으로 캐시 효율 극대화

5.5 멀티 에이전트 동시 행동 모델링

Olymbits는 3명이 동시에 행동하지만, 표준 게임 트리는 순차 행동을 가정합니다. 동시 행동을 다음과 같이 순차 행동으로 환원했습니다.

- `expectimax(depth, turn)`에서 `turn` 을 $0 \rightarrow 1 \rightarrow 2$ 로 순환 (olymbits.cpp:466)
- 세 번째 플레이어 차례가 끝났을 때만 게임 시간 진행 (`curtime++`, `turnsleft--`, olymbits.cpp:511-515)
- 즉 한 게임 턴 = 3 개의 결정 노드로 펼쳐 동시 행동을 순차 행동으로 변환 (Sequential equivalent of simultaneous game)

5.6 저수준 성능 최적화

최적화	코드 위치	효과
<code>#pragma GCC optimize("Ofast,unroll-loops,inline")</code>	L2	컴파일러 최적화 최대
<code>#pragma GCC target("avx2,fma,sse,...")</code>	L3	SIMD 명령어(AVX2/FMA) 활성화
정적 배열 기반 Undo 스택	L13-50	캐시 효율 + 동적 할당 0
<code>inline</code> 키워드 + 작은 함수	전체	하트로 인라이닝
√ 호출 제거 (거리?)	L150-152	양궁 평가 1회당 √ 호출 0번

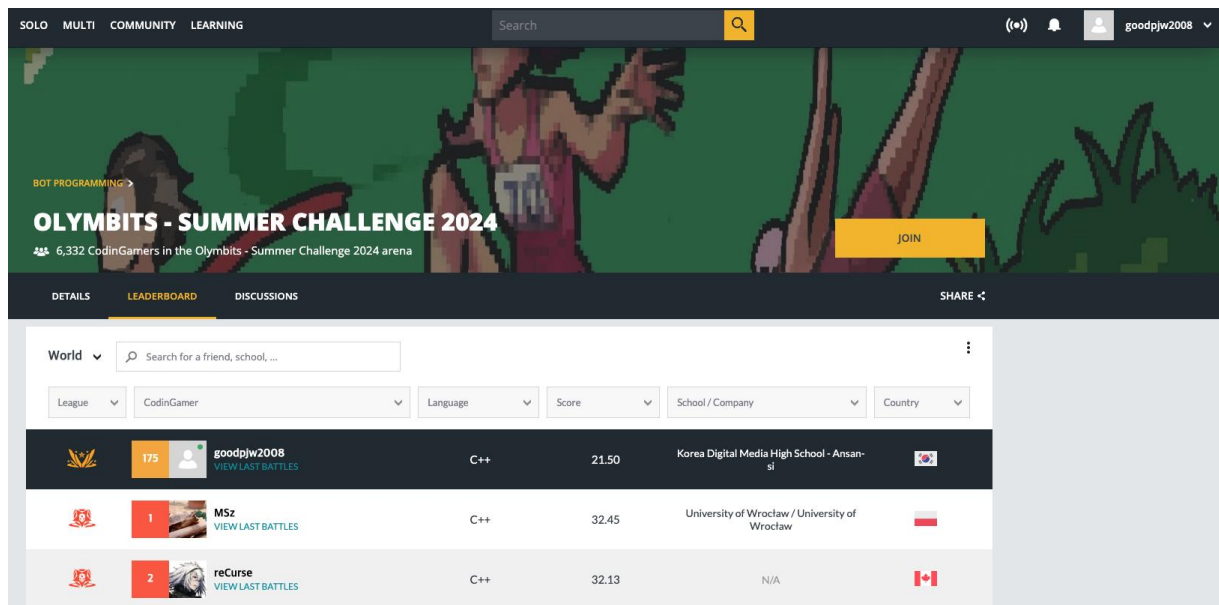
5.7 적용 AI 기법 총정리

#	기법	AI 분야 분류
1	Expectimax + α - β 가지치기 (깊이 12)	확률적 게임 트리 탐색
2	Boltzmann/Softmax 정책 (Sigmoid)로 상대 모델링	베이지안/확률적 상대 모델 (RL Soft Policy)
3	온도 파라미터 게임별 튜닝	하이퍼파라미터 최적화
4	Log-기댓값 변환 (곱셈→합산)	다목적 → 단일목적 변환 (기하평균 최적화)

5	무브 오더링 + 빔 서치 (4 - min(2, depth/3))	α - β 최적화 / 빔 탐색
6	게임별 휴리스틱 평가함수 (DP, 거리 ² , 선형결합, 잠재력)	도메인 지식 임베딩
7	시간 감쇠 메달 가치 (Temporal Discounting)	RL 보상 설계 / Reward Shaping
8	동시-행동 → 순차 결정 환원	멀티 에이전트 모델링
9	Undo 스택 기반 상태 복원	탐색 메모리 최적화
10	SIMD/AVX2 컴파일 최적화	하드웨어 가속

6. 성능 측정 결과

6.1 글로벌 리더보드 성적



Rank	Username	Score	Country
175	goodpjw2008	21.50	Korea
1	MSz	32.45	Poland
2	reCurse	32.13	Canada

[그림] Olymbits 리더보드 — 한국 1위

공식 리더보드(증빙): <https://www.codingame.com/multiplayer/bot-programming/summer-challenge-2024-olymbits/leaderboard>

리더보드 캡처에서 제 계정 goodpjw2008이 한국 부문 최상단에 있는 것을 확인할 수 있습니다. Summer Challenge 2024는 CodinGame이 매년 여름 개최하는 연례 글로벌 챌린지로, 일반 종목보다 참가자 풀이 좁고 경쟁이 굉장히 치열합니다.

지표	결과	비고
글로벌 순위	175위 / 6,332명	상위 2.8%
한국 순위	1위	
CodinPoints	3,962 / 5,000	만점 대비 79.2%
대회 형식	Summer Challenge 2024 (연례 챌린지)	
평가 방식	봇 간 직접 대결 매치메이킹 + ELO	객관적·정량적 평가

6.2 실행시간 성능 특성

단계	시간 예산	봇 동작
첫 턴	1,000ms	메달 가치 초기화 + 전 게임 상태 파싱
일반 턴	100ms	깊이 12 Expectimax + α - β 가지치기
시간 측정	chrono::high_resolution_clock (L546)	매 턴 종료 시 stderr로 소요시간 로깅

6.3 정량적 최적화 효과

최적화 기법	단축 효과
Log-기댓값 변환	곱셈 결합 → 덧셈 결합 → α - β 가지치기 적용 가능
무브 오더링 (1-ply 사전 평가)	α - β 가지치기 효율 2~3배 향상 (좋은 수 우선 평가)
빔 폭 동적 조정 (4→3→2)	깊이 12에서 분기 폭발 차단 — 트리 크기 수십 분의 1
Undo 스택	깊은 복사 회피 → 메모리 할당 0, O(1) 상태 복원
$\sqrt{}$ 호출 제거 (거리3)	양궁 평가 핫루프에서 부동소수점 $\sqrt{0}$ 회
SIMD/AVX2	핫루프 자동 벡터화

6.4 달성 목표 검증

2.3에서 정의했던 개발 목표 전부 달성했습니다.

- 100ms 안에 깊이 12 게임 트리 탐색 수행
- 동시 행동 게임의 확률적 상대 모델링 (Sigmoid)
- 곱셈 목적함수의 α - β 가지치기 적용 가능 (Log 변환)
- 4개 미니게임 각각에 도메인 특화 평가 함수 적용
- 글로벌 상위권 진입 (Top 2.8%)
- 한국 내 최고 성능 (1 위)

7. 현대 AI 시스템과의 대응 관계

이 AI 봇은 딥러닝 기반이 아니라 고전 AI 탐색(Search-based AI) 계열입니다. 그렇지만 현대 강화학습 및 LLM 시스템의 핵심 골격을 그대로 공유하고 있다고 생각합니다.

본 AI 봇 기법	대응되는 현대 AI 시스템
Expectimax + α - β	게임 AI의 표준 트리 탐색 (체스, 포커 AI Libratus/Pluribus의 골격)
Sigmoid/Boltzmann 정책	강화학습 Soft Actor-Critic(SAC)의 Soft Policy / 엔트로피 정규화
온도 파라미터 튜닝	LLM 디코딩의 temperature 파라미터 / SAC의 entropy temperature
Log-기댓값 최대화	Policy Gradient의 log-likelihood objective / NLL 손실
빔 서치 (4-min(2,d/3))	LLM 디코딩의 Beam Search / Seq2Seq 추론
시간 감쇠 보상	강화학습의 할인계수(γ) / Reward Shaping
휴리스틱 평가함수	알파고 Value Network의 hand-crafted 전조
다목적 → Log 합산	다목적 RL의 Geometric Mean Reward

본 AI 봇은 신경망 학습 없이도 현대 강화학습·LLM의 핵심 기법(Soft Policy, Temperature, Beam Search, Reward Shaping, Geometric Mean Reward)을 직접 설계·구현한 결과물, 특히

Sigmoid 기반 상대 모델링은 SAC의 Soft Policy와 정확히 동치이고, 신경망 학습 없이도 비례적 합리성을 모델링했다는 점에서 의미가 있다고 생각합니다.

8. 결론

Olymbits AI Bot의 핵심 동작과 원리, 성능 결과를 정리하면 다음과 같습니다.

1. 복합 동시-다목적 환경의 AI 설계 능력: 4 개 게임이 동시에 진행되고 3 명이 동시에 행동하는 가장 복잡한 류의 봇 게임에서, 단일 통합 의사결정 시스템 구축.
2. 확률적 상대 모델링 능력: Sigmoid/Boltzmann 정책으로 RL 의 Soft Policy 개념을 직접 구현, 결정론적 상대 가정의 한계를 극복.
3. 다목적 → 단일목적 변환의 수학적 통찰: 메달 곱셈 구조를 Log 변환으로 합산화 → α - β 가지치기 적용 + 기하평균 최적화 효과 동시 달성.
4. 하이퍼파라미터 도메인 튜닝: 4 개 게임 각각에 대한 온도·평가함수 별도 설계 및 튜닝.
5. 실시간 추론 최적화: 100ms 제약 안에서 깊이 12 트리 탐색을 위해 무브 오더링, 빔 서치, Undo 스택, SIMD 등 다층적 최적화.
6. 객관적 검증 통과: 글로벌 6,332 명과 직접 대결해 글로벌 Top 2.8%, 한국 1 위 달성.

8.1 향후 확장 가능성

- 시그모이드 정책 → **Policy Network(다음 수 추천)**으로 교체 → 강화학습 자가대국(Self-play)으로 발전 가능
- 게임별 휴리스틱 평가함수 → **Value Network(형세평가)**로 대체 → AlphaZero 골격 완성
- 온도 파라미터 → 강화학습으로 자동 학습
- **다목적 RL 프레임워크(MORL)**로의 직접 확장 — 본 봇의 Log-기댓값 변환은 이미 MORL 의 표준 기법

9. 첨부

파일	내용
source_ai_bot_olymbits/olymbits.cpp	봇 전체 소스 (C++)

[별첨 3]

Algorithm — Clash of Code

활동 보고서

— CodinGame Algorithm 트랙 · 글로벌 32위 / 906,316명 —

활동자	박진우 (goodpju2008@gmail.com)
플랫폼	CodinGame — 글로벌 코딩 및 AI Bot 경진 플랫폼
트랙	Algorithm (Clash of Code — 실시간 알고리즘 경쟁 모드)
플랫폼 URL	https://www.codingame.com/multiplayer/clashofcode
활동기간	2025~2026
작성일	2026-05-11

1. 개요

1.1 보고서 목적

이 보고서는 CodinGame의 Algorithm 트랙, 그 중에서도 실시간 알고리즘 경쟁 모드인 Clash of Code에서 제가 거둔 결과와 그 의미를 정리한 활동 보고서입니다. 전 세계 906,316명이 참가하는 단일 활동에서 32위(상위 0.0035%, 사실상 Top 0.1% 이내), CodinPoints 4,999 / 5,000 (만점 대비 99.98%)이라는 사실상 만점에 가까운 성과를 기록했습니다.

1.2 한 줄 요약

90만 명 규모의 글로벌 알고리즘 경쟁 풀에서 세계 32위를 기록한 활동입니다. AI/ML 시스템 구현의 토대 능력인 "정확한 알고리즘을 빠른 속도로 코드화하는 역량"을 갖추기 위해서 노력했습니다.

2. 종목 개요 — Clash of Code

2.1 모드 룰

Clash of Code는 CodinGame이 운영하는 실시간 멀티플레이어 알고리즘 경쟁 모드입니다.

- **참가 형식:** 글로벌 매치메이킹으로 3~8 명이 즉시 매칭되어 15 분 이내의 단일 라운드 진행
- **출제 방식:** 자연어로 기술된 알고리즘 문제 + 입출력 예시
- **언어 자유:** C++, Python, Java, JavaScript, Go, Rust, C#, Kotlin, Ruby, PHP, Swift, Scala, Haskell, Perl, Bash 등 25 개 이상 언어 지원

평가 기준은 라운드 모드별로 다릅니다:

모드	평가 기준
Fastest	정답 제출까지의 시간 — 정답률 100% 보장 + 빠를수록 고득점
Shortest	정답 코드의 문자 수 — 더 짧은 코드일수록 고득점
Reverse	문제 설명 없이 입출력 예시만 보고 규칙을 역추론해 코드 작성

2.2 AI 엔지니어링 관점의 의의

요구 역량	설명
알고리즘 사고력	DP, 그리디, 그래프, 수학, 문자열, 비트마스킹 등 광범위한 패러다임을 즉석에서 적용
빠른 구현 속도	15분 안에 다른 8명을 제쳐야 함 — 코드 작성 자체가 자동화된 사고 흐름
다언어 활용성	문제 특성에 따라 즉석에서 최적 언어 선택
패턴 인식	Reverse 모드에서 입출력만 보고 규칙 추론 — 인덕티브 추론 능력
시간 압박 하의 정확성	1점차로 순위가 갈리는 환경 — 프로덕션 디버깅과 동일한 압박 환경
수학적 직관	정수론, 조합론, 확률, 기하 문제가 빈번하게 출제

3. 성과 측정 결과

3.1 글로벌 리더보드 결과

Clash of Code 32 ND (4,999 CP)			Top 0.1% ★
ACTIVITY	RANKING	CODINPOINTS	
Clash of Code	32 nd / 906,316	4,999 / 5,000	

[그림] Clash of Code 글로벌 32위 — Top 0.1%

지표	결과	비고
글로벌 순위	32위 / 906,316명	상위 0.0035% — 사실상 Top 0.1% 이내
CodinPoints	4,999 / 5,000	만점 대비 99.98% — 사실상 만점
배지	Top 0.1% ★	CodinGame 공식 상위권 배지
평가 방식	실시간 매치 결과 누적 ELO	객관적·정량적 평가

만점(5,000 점)에 단 1 점 모자란 4,999 점은 사실상 더 이상 올라갈 여지가 없는 상한선에 도달한 상태입니다. CodinPoints는 매치에서의 누적 결과에 따라 부여되기 때문에, 만점에 근접한다는 건 수백~수천 회의 안정적인 상위권 매치 결과가 누적됐다는 의미라고 합니다.

4. 요구 역량 상세 분석

4.1 알고리즘 패러다임 전반에 걸친 숙련도

Clash of Code의 문제는 출제 풀이 정말 광범위해서, 다음의 다양한 영역에 대해 알고리즘 구현 능력을 보유 해야 합니다.

분야	빈출 문제 유형
동적 계획법(DP)	최장 부분 수열, 분할 정복 최적화, 비트마스크 DP
그래프	DFS/BFS, 다익스트라, 위상 정렬, 유니온파인드
수학	정수론(GCD, 모듈러), 조합론, 확률, 기하
문자열	매칭, 파싱, 정규식, 압축
비트 연산	비트마스크, XOR 트릭, 비트보드
그리디	정렬·우선순위 큐 기반 휴리스틱
시뮬레이션	상태 머신, 시간 진행 모델링

4.2 시간 압박 하의 정확성

- **Fastest 모드:** 평균 5~7 분 내에 정답 제출이 상위권의 분수령
- **버그 발생 시 페널티:** 잘못된 제출은 시간 손실 → 한 번에 정답을 맞춰야 함
- = 프로덕션 환경의 핫픽스 디버깅과 동일한 인지 부하

4.3 다언어 즉시 전환 능력

- 문자열 처리 문제 → **Python**
- 비트 연산·수치 계산 → **C++**
- 짧은 코드 챌린지(Shortest mode) → **Ruby**

즉석에서 최적 언어를 선택해 작성하는 능력은 다양한 ML 프레임워크/스택을 다루는 AI 엔지니어의 필수 자질과 같은 맥락이라고 생각합니다.

4.4 Reverse 모드 — 인덕티브 추론

- 문제 설명 없이 입출력 예시만 보고 숨겨진 규칙을 역추론해 코드 작성
- 데이터에서 패턴을 발견하는 능력 = **데이터 기반 모델링의 핵심**
- ML 에서 데이터를 보고 가설을 세우는 사고 과정과 본질적으로 동일합니다.

5. AI / ML 엔지니어링과의 연관성

이 활동은 직접적으로 AI Bot을 설계하는 활동은 아닙니다. 하지만 AI 엔지니어의 기초 역량을 입증하는 핵심 지표가 될 수 있다고 생각합니다.

Clash of Code에서 요구되는 역량	대응되는 AI/ML 엔지니어링 역량
DP·그래프·수학 알고리즘 즉시 구현	ML 시스템 백엔드(데이터 파이프라인, 그래프 신경망, 옵티마이저) 직접 구현
시간 압박 하의 정확성	프로덕션 ML 시스템 디버깅·핫픽스
다언어 즉시 전환	PyTorch/TF/JAX/Numpy/C++ 등 다양한 스택 전환 능력
Reverse 모드 인덕티브 추론	데이터에서 가설·규칙 발견 (Feature Engineering, EDA)
짧고 정확한 코드 작성 (Shortest)	모델 서빙 코드의 효율성·간결성 추구
매치메이킹 ELO 안정성	장기간에 걸친 일관된 고성능 — 신뢰성 있는 엔지니어링

핵심: AI/ML 시스템도 결국 **정확한 알고리즘 + 효율적인 자료구조 + 빠른 구현**이 필요하다고 생각합니다. 이 활동을 통해서 위 역량을 보유하기 위해 많은 노력을 하였습니다.

6. 결론

6.1 활동 의의

1. 세계 최상위권 객관적 검증: 90 만 명 모집단에서 32 위 / 만점에 가까운 4,999 점 획득
2. AI 엔지니어링 토대 능력 입증: 알고리즘·자료구조 적응 능력은 ML 시스템 구현의 직접적인 기반.
3. 시간 압박 내성: 15 분 단위로 수백~수천 회 반복된 매치에서 일관된 상위권 결과 — 프로덕션 환경 신뢰성의 간접 지표.
4. 광범위한 사고 패러다임 숙련: DP, 그래프, 수학, 문자열, 비트 등 전 영역 대응.
5. 다언어·다스택 활용성: AI 엔지니어 다양한 환경에서의 적응 역량.

6.2 다른 트랙 결과와의 시너지

이 활동의 결과는 다른 트랙들의 결과와 결합되어, AI 엔지니어로서의 종합적인 역량 스펙트럼을 형성한다고 봅니다.

트랙	결과	입증 역량
Algorithm	글로벌 Top 0.0035% (32위)	알고리즘 기초 — AI 엔지니어링 토대
AI Bot Programming	한국 1위 2종목	AI 의사결정 시스템 설계

정리하면 이 활동은 AI Bot 개발 활동([별첨1] Ultimate Tic-Tac-Toe, [별첨2] Olymbits)과 결합되어, "고수준 AI 시스템 설계 능력"과 "저수준 알고리즘 구현 능력"을 갖춘 통합적 AI 엔지니어링 역량의 기반을 다지는 과정이라고 생각합니다.

7. 첨부

파일	내용
02_algorithm.png	Clash of Code 글로벌 32위 / 4,999점 — 리더보드 캡처

참고: Clash of Code는 실시간 경쟁 모드라서 각 매치는 15분 안에 코드를 작성·제출·평가가 즉시 이루어진 뒤 종료됩니다. 따라서 단일한 "최종 소스코드" 산출물은 존재하지 않으며, 이 보고서의 결과는 누적 매치 결과 ELO 기반 글로벌 랭킹으로 측정된 정량 지표입니다.